Chapter 1: Deducing Types

Item 1: Understand template type deduction

For `template<typename T> void f(ParamType param)` called with `f(expr)`, deduction follows three cases based on `ParamType`: **Case 1: ParamType is a reference or pointer (not universal)** Strip reference-ness from `expr`, then pattern-match `expr`'s type against `ParamType`.

```
template<typename T> void f(T& param);
int x = 27;
const int cx = x;
const int& rx = x;
f(x);   // T = int,       param is int&
f(cx);  // T = const int, param is const int&
f(rx);  // T = const int, param is const int&   (ref
stripped)
```

Case 2: ParamType is a universal reference (T&&) Lvalue argument gives `T = U&`, then reference collapsing makes `param` an lvalue ref. Rvalue argument gives `T = U`, `param` is an rvalue ref.

```
template<typename T> void f(T&& param);
f(x);   // T = int&,  param is int&    (lvalue case)
f(27);  // T = int,   param is int&&   (rvalue case)
```

Case 3: ParamType is by-value Strip reference-ness, then strip top-level `const` and `volatile`.

```
template<typename T> void f(T param);
f(rx);  // T = int, param is int  (const and ref both
dropped)
```

Array and function decay A by-value parameter makes an array argument decay to a pointer to its first element; a by-reference parameter deduces the true array type (including size). The same machinery applies to function arguments.

```
template<typename T> void f(T  param);  // by value
template<typename T> void g(T& param);  // by reference
const char name[] = "abc";
f(name);  // T = const char*        (decays)
g(name);  // T = const char[4]      (true array type,
size kept)
```

Takeaway: Reference collapsing: `& &`, `& &&`, `&& &` all collapse to `&`. Only `&& &&` stays `&&`. Reference arguments are treated as non-references for matching; by-value params strip `const` / `volatile`; arrays and functions decay to pointers unless bound to a reference.

Item 2: Understand auto type deduction

`auto` deduction is template deduction with `auto` playing the role of `T`, with one exception: a braced initializer is deduced as `std::initializer_list<T>`.

```
auto x = 27;       // int
auto x2(27);       // int
auto x3 = { 27 };  // std::initializer_list<int>
auto x4{ 27 };     // C++17: int. Pre-C++17:
initializer_list<int>.
```

`auto` return types (C++14) and `auto` lambda parameters use **template** deduction, not `auto` deduction, so a braced initializer there is an error rather than an `initializer_list`.

Pitfall: `auto x{1, 2, 3}` is still an `initializer_list` even in C++17. Direct-init with a single element became `int` in C++17, but multi-element braced init stays an `initializer_list`.

Takeaway: `auto` deduction is template deduction except a braced initializer is assumed to be a `std::initializer_list`. `auto` in a function return type or lambda parameter uses template deduction, which rejects braced initializers.

Item 3: Understand `decltype`

`decltype` reports the declared type of a name or expression, almost always **without** modification, unlike template/`auto` deduction which strips references and cv-qualifiers. Primary use: declaring a return type that depends on the parameter types.

```
// C++11: trailing return type lets params appear in the
return type
template<typename Container, typename Index>
auto authAndAccess(Container& c, Index i)
  -> decltype(c[i])
{
  authenticateUser();
  return c[i];
}
```

In C++14 the trailing `-> decltype(c[i])` can be dropped, but plain `auto` uses template deduction and strips the reference: `c[i]` returns `T&` yet `auto` deduces `T`, returning an rvalue. Use `decltype(auto)` to preserve it.

```
// C++14: preserves reference-ness of c[i]
template<typename Container, typename Index>
decltype(auto) authAndAccess(Container& c, Index i)
{
  authenticateUser();
  return c[i];   // returns exactly c[i]'s type, e.g.
int&
}
```

The name-vs-expression distinction is the classic trap: a bare name yields its declared type, but any other lvalue expression of type `T` yields `T&`.

```
int i = 0;
decltype(i)   a;  // int
decltype((i)) b = i;  // int&  -- (i) is an expression,
not a name
```

Takeaway: `decltype` almost always yields the type of a variable or expression with no modification. For lvalue expressions of type `T` that are not just a name, it yields `T&`. `decltype(auto)` deduces like `auto` but using `decltype` rules.

Item 4: Know how to view deduced types

Three points in the toolchain reveal deduced types: the IDE editor (hover), a deliberate compiler error, and the Boost.TypeIndex library at runtime.

```
// Force an error that names the type: declare but never
define a template
template<typename T> class TD;   // TD = "Type
Displayer"
```

```
TD<decltype(x)> xType;           // error message prints
decltype(x)
TD<decltype(y)> yType;
```

Takeaway: Deduced types are visible via IDEs, forced compiler error messages, and Boost.TypeIndex. The first two can mislead (they may strip or add cv/ref qualifiers), so understanding the deduction rules remains essential.

Chapter 2: auto

Item 5: Prefer `auto` to explicit type declarations

`auto` variables take their type from the initializer, so they cannot be left uninitialized and cannot suffer silent type mismatches. For closures, an `auto` variable holds the closure in its own exact type, so the compiler can size it precisely; storing the same closure in a `std::function` costs a fixed-size object that often heap-allocates and calls indirectly, making it bigger and slower.

```
auto deref = [](const auto& p1, const auto& p2)
             { return *p1 < *p2; };          // exact
closure type, no overhead

std::function<bool(const WidgetPtr&,
                   const WidgetPtr&)> derefF =
deref;  // bigger, slower
```

Takeaway: `auto` variables must be initialized, resist portability/efficiency type mismatches, ease refactoring, and need less typing than explicit types. They are subject to the pitfalls in Items 2 and 6.

Item 6: Use the explicitly typed initializer idiom when `auto` deduces undesired types

`std::vector<bool>` does not store real `bool`s; it is a packed-bit specialization whose `operator[]` returns a proxy object (`std::vector<bool>::reference`), not a `bool&`. Assigning to a `bool` invokes the proxy's implicit conversion and works correctly. With `auto`, the variable **becomes** the proxy, which holds a pointer and bit offset into a temporary vector that is destroyed at end of statement, leaving a dangling proxy and undefined behavior on later use.

```
std::vector<bool> features(const Widget& w);

bool highPriority = features(w)[5];  // proxy -> bool,
fine
auto highPriority = features(w)[5];  // proxy dangles ->
UB
```

The fix is the **explicitly typed initializer idiom**: keep `auto`, but `static_cast` the proxy expression to the type you actually want.

```
auto highPriority = static_cast<bool>(features(w)
[5]);  // forces bool now
```

Takeaway: Invisible proxy types can make `auto` deduce the "wrong" type for an initializer. The explicitly typed initializer idiom (`auto x = static_cast<T>(...)`) forces `auto` to deduce the type you intend.

Chapter 3: Moving to Modern C++

Item 7: Distinguish between `()` and `{}` when creating objects

C++11 adds braced (uniform) initialization alongside parentheses and `=`. Braces can specify container contents, forbid narrowing conversions, and sidestep the most vexing parse.

```
int a(0);     // parentheses
int b = 0;    // "="
int c{0};     // braces
int d = {0};  // treated as braces

double x = 1.5;
int n1{x};    // error: narrowing conversion
int n2(x);    // OK, silently truncates to 1
```

Most vexing parse `Widget w1();` declares a **function** returning `Widget`, not an object. Braces remove the ambiguity.

```
Widget w1();  // function declaration (the vexing parse)
Widget w2{};  // default-constructed object
```

initializer_list dominates overload resolution If a class has a `std::initializer_list` constructor, braced init strongly prefers it, even over better-matching constructors, and even when it forces narrowing or odd conversions. It falls back to ordinary constructors only when no conversion to the list's element type is possible.

```
class Widget {
public:
  Widget(int i, bool b);
  Widget(std::initializer_list<long double> il);
};

Widget w3(10, true);  // (int, bool) ctor
Widget w4{10, true};  // init_list ctor; 10 and true ->
long double
```

Empty braces mean default construction, not an empty list. To call the list constructor with an empty list, nest the braces.

```
Widget w5{};    // default ctor (NOT empty
initializer_list)
Widget w6({});  // init_list ctor, empty list
Widget w7{{}};  // init_list ctor, empty list
```

Takeaway: Braced init is the most broadly usable syntax: it prevents narrowing and is immune to the most vexing parse. In overload resolution, braces match an `initializer_list` constructor whenever possible, even over better matches (`std::vector` is the classic example). Choosing `()` vs `{}` inside templates is genuinely hard, since the author cannot know which the caller intends.

Item 8: Prefer `nullptr` to `0` and `NULL`

`0` is an `int`, and `NULL` is some implementation-defined integral type, so neither is genuinely a pointer. When a function is overloaded on integral and pointer types, passing `0` or `NULL` resolves to the integral overload, not the pointer one, which is rarely what you want. `nullptr` has type `std::nullptr_t`, which implicitly converts to every pointer type but to no integral type, so it always resolves to the pointer overload.

```
void f(int);
void f(void*);

f(0);       // calls f(int)
f(NULL);    // calls f(int)  (usually)
f(nullptr); // calls f(void*) -- unambiguous
```

Takeaway: Prefer `nullptr` to `0` and `NULL`. Avoid overloading on integral and pointer types, since `0`/`NULL` make such calls ambiguous or wrong.

Item 9: Prefer alias declarations to `typedef`s

A `using` alias does everything a `typedef` does and can additionally be templatized (an **alias template**), which a `typedef` cannot. Alias templates also drop the `::type` suffix and, inside templates, the `typename` prefix that dependent `typedef`s require.

```
// alias template -- not possible with typedef
template<typename T>
using MyAlloc = std::vector<T, MyAllocator<T>>;
MyAlloc<int> v;   // clean

// C++11 trait: needs ::type and, when dependent,
typename
typename std::remove_const<T>::type   x;
// C++14 alias-template form: no ::type, no typename
std::remove_const_t<T>                y;
```

Takeaway: `typedef`s do not support templatization; alias declarations do. Alias templates avoid `::type` and the `typename` prefix often needed for dependent `typedef`s. C++14 provides `_t` alias templates for all the C++11 type traits.

Item 10: Prefer scoped `enum`s to unscoped `enum`s

An unscoped `enum { ... }` leaks its enumerator names into the enclosing scope and lets them implicitly convert to the underlying integral type. A scoped `enum class { ... }` keeps names scoped (`Enum::name`) and blocks implicit conversion, requiring an explicit `static_cast`.

```
enum Color { black, white, red };      // unscoped:
leaks names
auto white = false;                    // error: 'white'
already in scope
if (red < 14.5) { /* ... */ }          // implicit
convert to int: compiles

enum class Color { black, white, red }; // scoped
Color c = Color::white;                  // must qualify
if (static_cast<double>(c) < 14.5) { }  // no implicit
convert
```

Scoped enums have a default underlying type of `int` and can always be forward declared. Unscoped enums have an unspecified underlying type, so they can be forward declared only if you specify one explicitly.

```
enum class Status;                 // OK: scoped,
defaults to int
enum class Status: std::uint32_t;  // OK: explicit
underlying type
```

```
enum Color: std::uint8_t;          // OK only because
type is specified
```

One legitimate use for unscoped enums is naming `std::tuple` fields, where the implicit conversion to `std::size_t` is exactly what `std::get` wants.

```
using UserInfo = std::tuple<std::string, std::string,
std::size_t>;
enum UserInfoFields { uiName, uiEmail, uiReputation };
auto val = std::get<uiEmail>(uInfo);  // reads better
than std::get<1>
```

For scoped enums, a `constexpr` template recovers the same brevity by converting an enumerator to its underlying type at compile time.

```
template<typename E>                    // C++14
constexpr auto toUType(E enumerator) noexcept
{
  return
static_cast<std::underlying_type_t<E>>(enumerator);
}
auto val =
std::get<toUType(UserInfoFields::uiEmail)>(uInfo);
```

Takeaway: C++98-style enums are now "unscoped." Scoped enumerators are visible only inside the enum and convert only via a cast. Both can specify an underlying type, but only scoped enums default to one (`int`), so unscoped enums need an explicit underlying type to be forward declared.

Item 11: Prefer deleted functions to `private` undefined ones

The C++98 trick of declaring an unwanted function `private` and leaving it undefined relies on a link-time error. `= delete` makes the function unusable at compile time with a clearer diagnostic. Declare deleted functions `public`: the compiler often checks accessibility before deleted-ness, so a `private` deleted function can produce a misleading "is private" error instead of "is deleted."

```
class Widget {
public:
  Widget(const Widget&)            = delete;  // no copy
ctor
  Widget& operator=(const Widget&) = delete;  // no copy
assign
};
```

Unlike the `private` trick, deletion works on any function, including free functions and specific template instantiations, letting you filter out unwanted argument types.

```
bool isLucky(int);
bool isLucky(char)   = delete;  // reject char
bool isLucky(double) = delete;  // reject double and
float

template<typename T> void processPointer(T* ptr);
template<> void processPointer<void>(void*) =
delete;  // ban void*
```

Takeaway: Prefer deleted functions to `private` undefined ones. Any function can be deleted, including non-member functions and template instantiations.

Item 12: Declare overriding functions `override`

For a derived function to override a base virtual, every one of a long list of conditions must hold, and any mismatch silently creates a **new** function instead of an override. The conditions: the base function is virtual; names match (except destructors); parameter types match; constness matches; return types and exception specifications are compatible; and (C++11) reference qualifiers match. The `override` keyword makes the compiler verify all of them and error if the function overrides nothing.

```
struct Base {
  virtual void mf1() const;
  virtual void mf2(int x);
  virtual void mf3() &;
};
struct Derived: Base {
  virtual void mf1() override;        // error: missing
const
  virtual void mf2(unsigned) override; // error: param
type differs
  virtual void mf3() && override;      // error: ref-
qualifier differs
};
```

Reference qualifiers restrict a member function by the value category of `*this`, the same way `&` / `&&` on a parameter restrict an argument: `&` allows calls on lvalues, `&&` on rvalues. This is independent of virtualness.

```
class Widget {
public:
  void doWork() &;   // only when *this is an lvalue
  void doWork() &&;  // only when *this is an rvalue
};
```

Takeaway: Declare overriding functions `override`. Member function reference qualifiers let you treat lvalue and rvalue `*this` differently.

Item 13: Prefer `const_iterator`s to `iterator`s

A `const_iterator` is the STL equivalent of a pointer-to-`const`: it points at elements it cannot modify. The general advice "use `const` whenever practical" applies, but C++98 made it impractical, since containers gave no easy way to get a `const_iterator` and many member functions accepted only `iterator`. C++11 fixes this with `cbegin` / `cend` member functions.

```
std::vector<int> v{1, 2, 3};
auto it = std::find(v.cbegin(), v.cend(), 2);  //
const_iterator
v.insert(it, 1998);                            // C++11:
insert takes const_iterator
```

For maximally generic code (templates, library utilities), prefer the **non-member** `std::begin` / `cbegin` /etc. over member functions, since not all containers (or arrays) expose members. C++14 adds the non-member `cbegin` family; C++11 shipped non-member `begin` / `end` but omitted the `c`-prefixed versions, so in C++11 you may need to supply your own.

```
// non-member cbegin usable in C++11: const container +
non-member begin
template <class C>
auto cbegin(const C& container) ->
decltype(std::begin(container))
```

```
{
  return std::begin(container);  // begin on a const
container yields const_iterator
}
```

Takeaway: Prefer `const_iterator`s to `iterator`s. In maximally generic code, prefer the non-member versions of `begin`, `end`, `rbegin`, etc. over their member-function counterparts.

Item 14: Declare functions `noexcept` if they won't emit exceptions

`noexcept` declares that a function will not let an exception escape; if one tries to, `std::terminate` is called. It is part of the interface, so callers may rely on it, and it enables optimizations the C++98 `throw()` form cannot. With `throw()` the runtime stack must stay fully unwindable; with `noexcept` the optimizer need not keep the stack unwindable nor destroy objects in reverse construction order when an exception propagates out, yielding better object code.

```
int f(int x) throw();   // C++98: caller-side unwinding
guaranteed
int f(int x) noexcept;  // C++11: most optimizable
```

The biggest payoff is move semantics: operations like `std::vector::push_back` upgrade from copy to move only if the move is `noexcept`, because they must preserve the strong exception guarantee. `swap` and memory deallocation benefit similarly. You can also make the guarantee conditional with `noexcept(expr)`.

```
template <class T, size_t N>
void swap(T (&a)[N], T (&b)[N])
  noexcept(noexcept(swap(*a, *b)));  // noexcept iff
element swap is
```

Reserve `noexcept` for functions with a **wide contract** (no preconditions that, if violated, give undefined behavior); functions with a narrow contract should not be `noexcept`, since a precondition check may need to throw. Do not contort a function just to earn the specification. Note destructors are implicitly `noexcept` unless a member's destructor is not. The compiler does not require called functions to be `noexcept`, since they may be legacy or C code.

Takeaway: `noexcept` is part of the interface, so callers may depend on it. `noexcept` functions are more optimizable, and it is especially valuable for move operations, `swap`, memory deallocation, and destructors. Most functions are exception-neutral, not `noexcept`.

Item 15: Use `constexpr` whenever possible

A `constexpr` **object** is `const` and has a value known at compile time, usable where the language demands a compile-time constant (array sizes, template non-type arguments, `enum` values). A `constexpr` **function** produces a compile-time result when all its arguments are known at compile time, and behaves as an ordinary runtime function otherwise, so one function serves both roles.

```
constexpr int pow(int base, int exp) noexcept  // C++14
body allowed
{
  int result = 1;
  for (int i = 0; i < exp; ++i) result *= base;
  return result;
}
constexpr int n = pow(3, 2);   // computed at compile
time
std::array<int, pow(3, 2)> a;  // OK: compile-time
context

int b = readFromConsole();
int r = pow(b, 2);             // same function, runs at
runtime
```

`constexpr` implies `const` for objects, but not vice versa. C++11 limited `constexpr` function bodies to essentially a single `return`; C++14 lifted that. User-defined types can be literal types (usable in `constexpr`) if their constructors and relevant member functions are themselves `constexpr`. Because `constexpr` widens the contexts in which an object or function may be used, and it is part of the interface, removing it later can break callers, so apply it only where the value or computation is genuinely fixed.

Takeaway: `constexpr` objects are `const` and initialized with compile-time values. `constexpr` functions produce compile-time results when called with compile-time arguments. Both can be used in more contexts than non-`constexpr` counterparts, and `constexpr` is part of an object's or function's interface.

Item 16: Make `const` member functions thread-safe

A `const` member function promises not to change the object's logical state, but callers reasonably assume two threads can invoke it concurrently without synchronization. If such a function mutates a `mutable` data member (e.g. caching a computed result), that mutation is a data race. Guard a single mutable member with `std::atomic`; guard two or more that must stay mutually consistent with a `std::mutex` (declared `mutable`, which also makes the object non-copyable since `mutex` is move-only).

```
class Polynomial {
public:
  using RootsType = std::vector<double>;
  RootsType roots() const {
    std::lock_guard<std::mutex> g(m);   // multiple
members -> mutex
    if (!rootsValid) { /* compute, fill rootVals */
rootsValid = true; }
    return rootVals;
  }
private:
  mutable std::mutex  m;
  mutable bool        rootsValid{false};
  mutable RootsType   rootVals{};
};
```

Takeaway: Make `const` member functions thread-safe unless you are certain they will never be used in a concurrent context. `std::atomic` may outperform a mutex, but it suits only a single variable or memory location.

Item 17: Understand special member function generation

The special member functions the compiler may generate are: default constructor, destructor, copy constructor, copy assignment operator, and (C++11) the move constructor and move assignment operator. The move operations perform memberwise

moves, which are true moves only for members that support moving and fall back to copies otherwise.

```
class Widget {
public:
  Widget(Widget&& rhs);             // move ctor
  Widget& operator=(Widget&& rhs);  // move assignment
};
```

The two copy operations are independent: declaring one does not stop generation of the other. The two move operations are not independent: declaring either one stops generation of **both**. Furthermore, declaring any copy operation disables move generation, and declaring any move operation disables (deletes) the copy operations, the reasoning being that if memberwise copy/move is wrong for one, it is likely wrong for the other.

```
class Widget {
public:
  ~Widget();   // user-declared dtor...
  // ...so the compiler does NOT generate move
operations.
  // Copy ops still generated (but their generation is
deprecated here).
};
```

The "Rule of Three" (if you declare a copy ctor, copy assignment, or destructor, declare all three) interacts with this: because a user-declared destructor suppresses move generation, classes relying on a destructor silently get copies where moves were expected, which compiles but runs slower. Move operations are generated only when the class declares no copy operations, no move operations, and no destructor. Use `= default` to ask for the standard behavior explicitly.

```
class Widget {
public:
  ~Widget();                              // still want
default copies + moves
  Widget(const Widget&)            = default;
  Widget& operator=(const Widget&) = default;
  Widget(Widget&&)                 = default;
  Widget& operator=(Widget&&)      = default;
};
```

Member function **templates** never suppress generation of any special member function, regardless of what they could instantiate to.

Pitfall: A user-declared destructor silently disables move generation. The class still compiles, but operations that should move will copy instead, often a large and quiet performance loss. Declare moves `= default` if you want them back.

Takeaway: Special member functions are those the compiler may generate: default ctor, dtor, copy operations, and move operations. Move operations are generated only for classes lacking user-declared copy operations, move operations, or a destructor. The copy constructor is generated only absent a user-declared copy constructor and is deleted if a move operation is declared; copy assignment is analogous. Member function templates never suppress generation.

End-of-chapter note: `final` on a virtual function prevents overriding in derived classes; on a class, it prohibits use as a base class.

Chapter 4: Smart Pointers

Item 18: Use `std::unique_ptr` for exclusive-ownership resource management

`std::unique_ptr` models exclusive ownership: it is a move-only type (copying is disabled), and when it is destroyed or reassigned it destroys the resource it owns. It is as small and fast as a raw pointer by default, which makes it the right return type for factory functions.

```
std::unique_ptr<Widget> makeWidget();  // factory;
ownership moves to caller
auto w = makeWidget();
auto w2 = std::move(w);  // OK: ownership transferred, w
is now null
// auto w3 = w2;         // error: copy is deleted
```

The destruction mechanism is `delete` by default, but a custom deleter can be supplied as the second template parameter (and a deleter object at construction). Function-pointer or stateful function-object deleters enlarge the `unique_ptr` from one word to two or more; a capture-less lambda deleter adds no size.

```
auto delWidget = [](Widget* p){ makeLogEntry(p); delete
p; };  // no size cost
std::unique_ptr<Widget, decltype(delWidget)> uw(new
Widget, delWidget);
```

A `std::unique_ptr` converts implicitly and cheaply to a `std::shared_ptr`, which is why factories return `unique_ptr`: the caller decides the ownership model.

Takeaway: `std::unique_ptr` is a small, fast, move-only smart pointer for exclusive-ownership resource management. Destruction defaults to `delete`; custom deleters are supported, but stateful deleters and function-pointer deleters increase its size. Converting a `unique_ptr` to a `shared_ptr` is easy.

Item 19: Use `std::shared_ptr` for shared-ownership resource management

`std::shared_ptr` implements shared ownership via reference counting: construction increments the count, destruction decrements it, copy assignment does both, and the last `shared_ptr` to reach zero deletes the object. This costs performance. A `shared_ptr` is twice the size of a raw pointer (a pointer to the object and a pointer to a control block); the control block is dynamically allocated; and the reference-count manipulations must be atomic.

```
auto p1 = std::make_shared<Widget>();  // ref count 1
auto p2 = p1;                          // ref count 2
(atomic increment)
// p2 destroyed -> ref count 1; p1 destroyed -> ref
count 0 -> delete
auto p3 = std::move(p1);  // no count change: p1 nulled,
p3 takes over
```

Unlike `unique_ptr`, the deleter is not part of the `shared_ptr` type, only of its construction, so `shared_ptr<Widget>`s with different deleters share a type and can sit in one container. The control block holds the reference count, the weak count, and any custom deleter or allocator. A control block is created when you call `make_shared`, when you construct from a `unique_ptr`, or when you construct from a raw pointer; constructing two `shared_ptr`s from the **same** raw pointer creates two control blocks and leads to multiple deletions, which is undefined behavior.

```
auto raw = new Widget;
std::shared_ptr<Widget> sp1(raw);
std::shared_ptr<Widget> sp2(raw);  // UB: second control
block, double delete
```

To safely get a `shared_ptr` to the current object from a member function, derive from `std::enable_shared_from_this` and call `shared_from_this()`. There is no shared ownership back-conversion to `unique_ptr`, and no array API.

Takeaway: `std::shared_ptr`s offer convenience approaching garbage collection for shared lifetime management of arbitrary resources. They are typically twice the size of a raw pointer, incur control-block overhead, and require atomic reference-count manipulation. Custom deleters are supported without affecting the type. Avoid constructing more than one from a single raw pointer.

Item 20: Use `std::weak_ptr` for `std::shared_ptr`-like pointers that can dangle

A `std::weak_ptr` observes an object managed by `shared_ptr`s without contributing to its reference count, so the object can be deleted out from under it. A `weak_ptr` cannot be dereferenced directly; you check for expiry and obtain access atomically through `lock`, which returns a `shared_ptr` that is null if the object has expired. (Constructing a `shared_ptr` from an expired `weak_ptr` instead throws `std::bad_weak_ptr`.)

```
std::weak_ptr<Widget> wp = sp;        // does not raise
the ref count
if (auto locked = wp.lock()) {        //
shared_ptr<Widget>, null if expired
  locked->doWork();                   // safe: object
still alive here
} else {
  // object has been destroyed
}
```

It has the same overhead as a `shared_ptr` and maintains its own (weak) count in the control block. Typical uses are caches, observer lists, and breaking `shared_ptr` reference cycles (where two objects own each other and neither count can reach zero).

Takeaway: Use `std::weak_ptr` for `shared_ptr`-like pointers that can dangle. Potential uses include caching, observer lists, and the prevention of `std::shared_ptr` cycles.

Item 21: Prefer `std::make_unique` and `std::make_shared` to direct use of `new`

The make functions take arguments, perfectly forward them to the object's constructor, and return a smart pointer (`make_shared` in C++11, `make_unique` in C++14; `allocate_shared` is the third). Three reasons to prefer them. First, they remove source duplication: the type is named once, not

once for `new` and again for the smart-pointer type. Second, exception safety. In a call like `f(std::shared_ptr<Widget>(new Widget), computePriority())`, the compiler may order the `new`, the `shared_ptr` construction, and the `computePriority` call freely; if `computePriority` runs between the `new` and the `shared_ptr` construction and throws, the raw allocation leaks. `make_shared` fuses allocation and smart-pointer construction so there is no gap.

```
// leak-prone if computePriority() throws between new
and shared_ptr ctor
f(std::shared_ptr<Widget>(new Widget),
computePriority());
// safe: no raw pointer is ever exposed
f(std::make_shared<Widget>(), computePriority());
```

Third, `make_shared` performs a single allocation for the object and its control block together, versus two allocations with `new` (the object, then the control block), giving smaller, faster code. The make functions cannot be used when you need a custom deleter, and because they forward with parentheses they cannot directly do braced (`initializer_list`) initialization; the workaround is to build an `auto` `initializer_list` variable first and pass that.

```
auto initList = {10, 20};
auto spv = std::make_shared<std::vector<int>>(initList);
```

For `make_shared` specifically, two further cautions: the single fused allocation means the object's memory cannot be freed until the control block is also gone, so if `weak_ptr`s outlive the last `shared_ptr`, a large object's memory lingers; and classes with custom `operator new`/`delete` expect the object-sized allocation `new` would make, not the fused block. When you must use `new` for deleter compatibility, do the allocation in its own statement and `std::move` the resulting `shared_ptr` into the call to stay exception-safe and avoid an atomic count bump.

```
std::shared_ptr<Widget> sp(new Widget, customDeleter);
// ... configure sp safely ...
f(std::move(sp), computePriority());  // move avoids
atomic increment
```

Takeaway: Compared with direct use of `new`, make functions eliminate source duplication, improve exception safety, and (for `make_shared`/`allocate_shared`) produce smaller, faster code. They are inappropriate when you need a custom deleter or must pass braced initializers. For `shared_ptr`, additional ill-advised cases are classes with custom memory management, and systems where memory is tight, objects are very large, and `weak_ptr`s outlive the `shared_ptr`s.

Item 22: When using the Pimpl idiom, define special member functions in the implementation file

The Pimpl ("pointer to implementation") idiom moves a class's data members into a separately defined struct, leaving only a pointer to it in the class. Clients of the header avoid pulling in the headers those members need, cutting compile dependencies. C++98 used a raw pointer; the modern form uses `std::unique_ptr`. **Why naive `unique_ptr` Pimpl fails** With a raw pointer, the compiler-generated destructor's body does nothing meaningful (you manage the delete yourself). With `unique_ptr`, the destructor body calls the default deleter, which invokes `delete` on the pointee and therefore requires the pointee's type to be **complete** at the point the destructor is generated. If you let the compiler implicitly generate the destructor in the header, it is generated where `Impl` is only forward-declared and the code fails to compile. **Fix: declare in header, define in .cpp** Declare every special member function in the header and define it in the implementation file, where `Impl` is complete. `= default` is fine.

```
// widget.h
class Widget {
public:
  Widget();
  ~Widget();                                  //
declared, not defined here
  Widget(Widget&&);                           // same
for moves
  Widget& operator=(Widget&&);
  Widget(const Widget&);                      // and
copies if you want them
  Widget& operator=(const Widget&);
private:
  struct Impl;
  std::unique_ptr<Impl> pImpl;
};

// widget.cpp
struct Widget::Impl { /* real data members */ };
Widget::Widget() : pImpl(std::make_unique<Impl>()) {}
Widget::~Widget()                             =
default;  // OK here: Impl complete
Widget::Widget(Widget&&)                      = default;
Widget& Widget::operator=(Widget&&)           = default;
```

Why this does not apply to `shared_ptr` `shared_ptr`'s deleter is type-erased into the control block at construction time, not into the smart-pointer type, so the class can be destroyed without the destructor seeing `Impl`'s definition. Pimpl with `shared_ptr` works without the declare-in-header/define-in-cpp dance, at the usual `shared_ptr` overhead.

Takeaway: The Pimpl idiom decreases build times by reducing compilation dependencies between class clients and implementations. For `unique_ptr` Pimpl pointers, declare the special member functions in the class header and implement them in the implementation file, even if `= default` suffices. The advice applies to `unique_ptr` but not `shared_ptr`.

Chapter 5: Rvalue References, Move Semantics, and Perfect Forwarding

Item 23: Understand `std::move` and `std::forward`

Both are pure casts that emit no instructions. `std::move` casts its argument **unconditionally** to an rvalue (`T&&`). `std::forward` casts only when its argument was originally bound to an rvalue.

```
// C++14 implementation of std::move
template<typename T>
decltype(auto) move(T&& param) {
  using ReturnType = std::remove_reference_t<T>&&;
  return static_cast<ReturnType>(param);
}
```

The `remove_reference` is necessary: if `T` deduces to `Foo&` (lvalue argument), then `T&&` collapses to `Foo&` and you would accidentally return an lvalue reference, not the rvalue you wanted. The cast to rvalue makes an object **eligible** for moving, not guaranteed to move. A `const` rvalue cannot bind to a non-const move constructor's `T&&` parameter, so it falls through to the copy constructor's `const T&` parameter and silently copies.

```
const std::string cs = "hello";
std::string s2 = std::move(cs);  // compiles, but copies
(cs is const)
```

Pitfall: `std::move` on a `const` object silently calls the copy operation. If you want move semantics, do not make the source `const` .

Takeaway: `std::move` performs an unconditional cast to rvalue; by itself it moves nothing. `std::forward` casts to rvalue only when its argument is bound to an rvalue. Neither does anything at runtime.

Item 24: Distinguish universal references from rvalue references

`T&&` in source code has two meanings. It is an **rvalue reference** (binds only to rvalues, marks the object movable) in most contexts. It is a **universal reference** (binds to lvalues and rvalues, const and non-const, volatile and non-volatile) in exactly one configuration: the form is precisely `T&&` for a deducible template parameter, or `auto&&` , and type deduction is in play. Anything that breaks the exact form (a const qualifier, a wrapping type like `std::vector<T>&&`) makes it an ordinary rvalue reference.

```
void f(Widget&& param);              // rvalue ref: no
deduction
Widget&& var1 = makeWidget();        // rvalue ref: no
deduction

template<typename T>
void g(T&& param);                   // universal ref:
deduced T, exact form

auto&& var2 = expr;                  // universal ref:
deduced auto

template<typename T>
void h(std::vector<T>&& param);      // rvalue ref: not
exactly T&&

template<typename T>
void k(const T&& param);             // rvalue ref:
const breaks the form
```

`std::vector::emplace_back` is itself a function template, so its parameter is a universal reference. `push_back` 's parameter type is built from the **container's** `T` , not a fresh template parameter, so it is an ordinary rvalue reference. A universal reference resolves to an lvalue reference when initialized with an lvalue and to an rvalue reference when initialized with an rvalue. The mechanism underneath is reference collapsing (Item 28). The variadic forwarding lambda is the textbook use:

```
auto timer = [](auto&& func, auto&&... params) {
  // start clock
  std::forward<decltype(func)>(func)(
    std::forward<decltype(params)>(params)...);
  // stop clock
};
```

Takeaway: If a function template parameter has type `T&&` for a deduced `T` , or an object is declared `auto&&` , it is a universal reference. If the form is not precisely `T&&` / `auto&&` , or no deduction occurs, it is an rvalue reference. Universal references correspond to lvalue references when bound to lvalues and to rvalue references when bound to rvalues.

Item 25: Use `std::move` on rvalue references and `std::forward` on universal references

Rvalue references bind only to rvalues, which are always move-eligible, so an unconditional `std::move` is safe and correct. Universal references may be bound to either lvalues or rvalues; an unconditional `move` would silently steal from lvalues the caller still owns. `std::forward` casts to rvalue only in the rvalue case, so it is the right tool for universal references. Apply the cast on the **last** use of the parameter; earlier uses leave the object valid.

```
class Widget {
public:
  Widget(Widget&& rhs) : s(std::move(rhs.s)) { }   //
rvalue ref -> move
};

template<typename T>
void setName(T&& newName) {
  name = std::forward<T>(newName);                 //
universal ref -> forward
}
```

Same rule when returning by value: if the return expression is an rvalue ref or universal ref parameter, apply `move` / `forward` so the return value is moved, not copied.

```
Matrix operator+(Matrix&& lhs, const Matrix& rhs) {
  lhs += rhs;
  return std::move(lhs);   // moves into the return slot
}
```

Do not move local returns Do **not** apply `std::move` to a local variable in a `return` statement. The compiler is permitted to perform Return Value Optimization (RVO), constructing `w` directly in the caller's return slot, eliding any copy or move; failing that, the standard requires the compiler to treat the local as an rvalue, so it moves

implicitly. Writing `return std::move(w)` defeats RVO (since `std::move(w)`'s type does not match the return type's eligibility rules) without buying anything.

```
Widget makeWidget() {
  Widget w;
  // ... configure w ...
  return w;          // RVO eligible. Do NOT write
std::move(w).
}
```

Takeaway: Apply `std::move` to rvalue references and `std::forward` to universal references the last time each is used. Do the same for rvalue/universal references returned from by-value-returning functions. Never apply `std::move`/`std::forward` to local objects otherwise eligible for RVO.

Item 26: Avoid overloading on universal references

A universal-reference function template synthesizes an exact match for almost any argument type. In overload resolution, exact matches beat promotions and standard conversions, so the universal-reference overload tends to win against everything else. Overloading on universal references therefore hijacks calls you intended for other overloads.

```
template<typename T> void logAndAdd(T&& name);  //
universal ref overload
void logAndAdd(int idx);                        // int
overload

short s = 42;
logAndAdd(s);   // calls the template (T = short&,
exact)
                // NOT the int overload (would require
promotion)
```

Perfect-forwarding constructors are the worst case. The copy constructor takes `const Person&`; the template instantiates to take `Person&` for a non-const lvalue argument, which is a better match. So `Person p2(p1)` calls the template, not the copy ctor.

```
class Person {
public:
  template<typename T>
  explicit Person(T&& n);          // universal ref ctor
  Person(const Person& p);         // copy ctor
};
Person p1("Alice");
Person p2(p1);   // calls the template, not the copy
ctor
```

The same trap hijacks derived-class calls to base copy/move constructors that would otherwise pattern-match on the derived type.

Takeaway: Overloading on universal references almost always leads to the universal-reference overload being called more often than intended. Perfect-forwarding constructors are especially problematic: they typically beat copy constructors for non-const lvalues and can hijack derived-class calls to base copy/move constructors.

Item 27: Familiarize yourself with alternatives to overloading on universal references

Several alternatives let you keep the efficiency benefits without the hijacking. **Abandon overloading** Give the variants distinct names. Boring and effective. **Pass by `const T&`** The C++98 fallback. You give up the efficiency that motivated universal references in the first place. **Pass by value** Reasonable when the value will be copied or moved into storage anyway (Item 41). **Tag dispatch** Keep one universal-reference function template as the public entry point, but have it call an overloaded implementation function with a compile-time **type** tag indicating which case applies. The tag is a type (`std::true_type` / `std::false_type`), not a `bool`, because overload resolution dispatches on type at compile time.

```
template<typename T>
void logAndAdd(T&& name) {
  logAndAddImpl(
    std::forward<T>(name),
    std::is_integral<std::remove_reference_t<T>>{});  //
tag
}
template<typename T>
void logAndAddImpl(T&& name, std::false_type);  // non-
integral case
void logAndAddImpl(int idx,  std::true_type);   //
integral case
```

The `remove_reference_t` is essential: a deduced `T` for an lvalue argument is `int&`, which `std::is_integral` reports as **not** integral. **Constrain with `std::enable_if`** When the universal-reference overload is still too greedy (notably for constructors), use SFINAE to remove it from the overload set when its conditions fail. `std::decay_t` strips references, cv-qualifiers, and array/function-to-pointer decay, so the trait checks see the underlying type.

```
class Person {
public:
  template<typename T,
           typename = std::enable_if_t<
             !std::is_base_of_v<Person, std::decay_t<T>>
&&
             !
std::is_integral_v<std::remove_reference_t<T>>>>
  explicit Person(T&& n);

  explicit Person(int idx);
  // copy/move generated normally
};
```

C++20 concepts (`requires` clauses) replace this pattern with much cleaner syntax, but the SFINAE form is what you will see in pre-C++20 code.

Takeaway: Alternatives to combining universal references with overloading include distinct function names, pass-by-`const`-lvalue-reference, pass-by-value, and tag dispatch. Constraining templates with `std::enable_if` permits universal references and overloading together, controlling when compilers may pick the universal-reference overload. Universal-reference parameters often have efficiency advantages but usability disadvantages.

Item 28: Understand reference collapsing

You cannot write `int& &` in source, but the compiler may produce a reference-to-reference during template instantiation or alias substitution. The collapsing rule: if either reference is an lvalue

reference, the result is an lvalue reference; otherwise (both are rvalue references) the result is an rvalue reference. So `& &`, `& &&`, `&& &` all collapse to `&`; only `&& &&` stays `&&`. The template-deduction half of the trick: when an lvalue of type `T` is passed to a `T&&` parameter, the compiler deduces `T` as the **reference** type `T&`, not the bare type. Combined with collapsing, the parameter becomes an lvalue reference. When an rvalue is passed, `T` deduces as the bare type and the parameter stays `T&&`. So `T` records the value category of the argument, which is exactly what `std::forward<T>` reads to decide whether to cast.

```
template<typename T> void f(T&& param);
int x = 0;
f(x);   // T = int&; param's type is int& &&  -> int&
f(0);   // T = int;  param's type is int&&    -> int&&
```

Four contexts in which reference collapsing occurs: template instantiation, `auto` type deduction, `typedef`/alias-declaration substitution, and `decltype`.

Takeaway: When the compiler produces a reference-to-reference in any of the four collapsing contexts (template instantiation, `auto` deduction, `typedef`/alias substitution, `decltype`), the result is a single reference: lvalue if either was lvalue, rvalue only if both were rvalue. Universal references are rvalue references in contexts where type deduction distinguishes lvalues from rvalues **and** reference collapsing occurs.

Item 29: Assume that move operations are not present, not cheap, and not used

Move semantics is a major C++11 feature, but you cannot assume every object benefits. Several common situations defeat moves. **No move at all** Pre-C++11 types and types that delete or fail to provide moves will copy. **Move is no faster than copy** `std::array<T, N>` stores its elements in-place, so moving an array is element-by- element moves of all `N` items, the same cost as copying. Short strings under Small String Optimization (SSO) store data inline; moving them is a copy of the inline buffer plus a source reset, no faster than copying. **Move is unusable** Operations like `std::vector::push_back` upgrade from copy to move only if the element's move constructor is `noexcept` (Item 14), to preserve the strong exception guarantee. A non-`noexcept` move means push_back keeps copying. **The source is an lvalue** Without `std::move`, lvalues bind to copy operations, not move operations, regardless of whether the type supports moving.

Takeaway: Assume that move operations are not present, not cheap, and not used. In code with known types or guaranteed move-semantics support, the assumption is unnecessary.

Item 30: Familiarize yourself with perfect forwarding failure cases

Perfect forwarding fails when calling `f(arg)` does one thing but calling `fwd(arg)` (a universal-reference forwarder around `f`) does something different, either because type deduction fails or because it deduces a type that produces different behavior. **Braced initializers** The compiler is forbidden from deducing a template type for a brace-enclosed initializer list (except for an explicit `std::initializer_list` parameter).

```
void f(std::vector<int> v);
template<typename... Ts> void fwd(Ts&&... args);
f({1, 2, 3});     // OK
fwd({1, 2, 3});   // error: cannot deduce
auto il = {1, 2, 3};
fwd(il);          // OK: auto deduces
std::initializer_list<int>
```

`0` or `NULL` as a null pointer Both deduce to integral types, not pointer types. Forwarding them to a function expecting a pointer fails. Use `nullptr`. **Declaration-only integral `static const` members** An in-class initialized `static const` without an out-of-class definition has no address. By-value parameters do not need one; by-reference parameters do. Forwarding through a universal reference is a by-reference operation, so the linker error appears even though the direct call works.

```
struct Widget { static const std::size_t MinVals =
28; };  // declared, not defined
f(Widget::MinVals);    // OK: by-value, no address taken
fwd(Widget::MinVals);  // link error: no definition for
MinVals
```

Overloaded function names and template names There is no single type to deduce. Disambiguate manually by `static_cast`-ing to the desired function pointer type, or by explicitly specializing the template. **Bitfields** A non-`const` reference cannot bind to a bitfield, since bitfields have no addressable sub-byte location. By-value and `const`-reference parameters work because they bind through a temporary. Forwarding fails; copy into a local first.

Takeaway: Perfect forwarding fails when template type deduction fails or deduces the wrong type. The argument kinds that cause failure are braced initializers, `0`/`NULL` as null pointers, declaration-only integral `static const` data members, template and overloaded function names, and bitfields.

End-of-chapter note: "Universal reference" and "forwarding reference" are the same thing; the standard uses the latter. RVO refers to elision of **unnamed** (temporary) return values; NRVO refers to **named** local variables returned by value. Since C++17, RVO is mandatory for prvalue returns; NRVO remains permitted but not required.

Chapter 6: Lambda Expressions

A **lambda expression** is the source-level construct (`[capture](params){body}`). The runtime object it produces is a **closure**; depending on capture mode it holds copies of or references to the captured data. The class type from which the closure is instantiated is the **closure class**; each lambda makes the compiler synthesize a unique one, with the lambda body becoming the body of its `operator()`.

Item 31: Avoid default capture modes

The two default modes are `[&]` (capture all used locals by reference) and `[=]` (capture all used locals by value). Both have failure modes. **`[&]`: dangling references** If the closure outlives any captured local, the reference inside the closure dangles. The danger is not visible at the lambda's source location.

```
using FilterContainer =
std::vector<std::function<bool(int)>>;
FilterContainer filters;
void addDivisorFilter() {
  auto divisor = computeDivisor();
  filters.emplace_back(
    [&](int value) { return value % divisor ==
0; });  // dangles after return
}
```

`[=]`: misleading self-containment Captures only non-`static` local variables and parameters visible in the enclosing scope. Inside a member function, the data member you "captured" was actually `this`: `[=]` copies the `this` pointer, and the closure indirects through it. If the object is destroyed before the closure runs, the indirection dangles.

```
class Widget {
  int divisor;
  void addFilter() const {
    filters.emplace_back(
      [=](int value) { return value % divisor ==
0; });  // captures this, not divisor
  }
};
```

Two fixes. Pre-C++14, copy the member to a local first and capture that local. C++14 introduced **init capture** (Item 32), which lets you spell out a closure member and its initializer directly.

```
// C++11
auto d = divisor;
filters.emplace_back([d](int v){ return v % d == 0; });

// C++14
filters.emplace_back([divisor = divisor](int v){ return
v % divisor == 0; });
```

Objects with **static** storage duration (globals, function-static, class-static) cannot be captured but can be used inside a lambda. A `[=]` lambda that "uses" a static variable is still not self-contained: the static is read live each time, with whatever value it currently holds.

Takeaway: Default by-reference capture can leave dangling references. Default by-value capture is susceptible to dangling pointers (especially `this`) and misleadingly suggests the closure is self-contained.

Item 32: Use init capture to move objects into closures

C++11 captures are either by value (copy) or by reference; there is no way to move a move-only object into a closure or to elide an expensive copy. C++14's **init capture** (also called **generalized lambda capture**) lets you specify both the name of a member in the closure class and the expression that initializes it. The two sides of the `=` live in different scopes: the left-hand name is in the closure class's scope, the right-hand expression is in the enclosing scope.

```
auto pw = std::make_unique<Widget>();
// ... configure pw ...
auto func = [pw = std::move(pw)] {           // move
into the closure
  return pw->isValidated() && pw->isArchived();
};
```

In C++11 the same effect needs either a hand-written function object class or a `std::bind` whose bound object is a `unique_ptr` moved in, with the lambda reaching into the bind expression. Init capture is strictly better when available.

Takeaway: Use C++14 init capture to move objects into closures. In C++11, emulate it with hand-written classes or `std::bind`.

Item 33: Use `decltype` on `auto&&` parameters to `std::forward` them

A C++14 **generic lambda** uses `auto` (or `auto&&`) in its parameter list. The synthesized `operator()` becomes a template, with one template parameter per `auto` parameter. To preserve the value category of arguments through a generic lambda, declare parameters `auto&&` (universal references) and forward them with `std::forward<decltype(param)>(param)`.

```
auto f = [](auto&&... params) {
  return func(
normalize(std::forward<decltype(params)>(params)...));
};
```

`decltype(param)` yields `T&` for an lvalue argument and `T&&` for an rvalue argument, which is exactly the type `std::forward` expects. Instantiating `std::forward` with `T&&` produces the same result as instantiating it with the bare type `T`, so the rvalue case forwards correctly. Variadic generic lambdas extend the same pattern to packs.

Takeaway: Use `decltype` on `auto&&` lambda parameters to `std::forward` them.

Item 34: Prefer lambdas to `std::bind`

`std::bind` was the C++11 way to do partial application. C++14 lambdas supersede it on every axis worth caring about. **Readability** A lambda shows its call expression literally. A `std::bind` expression hides it behind placeholders (`_1`, `_2`) and demands you remember bind-time vs call-time evaluation rules.

```
auto setSoundL = [](Sound s) {
  setAlarm(steady_clock::now() + 1h, s, 30s);
};
```

```
using namespace std::placeholders;
auto setSoundB = std::bind(setAlarm,
  std::bind(std::plus<>(), steady_clock::now(), 1h),  //
bound at bind time
  _1, 30s);
```

Argument evaluation timing Arguments to `bind` are evaluated when `bind` runs, not when the resulting object is called. The `now() + 1h` above means "one hour after the bind expression evaluated," not "one hour after the closure was called." A lambda captures the expression `now() + 1h` inside its body, so it runs at call time. **Overloads** A lambda with the right parameter list resolves overloads naturally. `bind` must be told which overload to pick (via a `static_cast` to the desired function pointer type), since it cannot deduce from argument types alone. **Efficiency** `bind` stores arguments by value (use `std::ref` to opt out, mentioned in your end-of-chapter note) and its call operator forwards them. Its inner call goes through a function pointer, which compilers inline less reliably than a lambda's inline `operator()` body. Lambdas tend to produce smaller, faster code. The only C++11-era reasons to reach for `bind` were move capture and polymorphic function-call operators (binding to an object with a templated `operator()`). Both are non-issues in C++14 (init capture and generic lambdas).

Takeaway: Lambdas are more readable, more expressive, and may be more efficient than `std::bind`. In C++11 only, `bind` is useful for emulating move capture or for binding objects with templatized function-call operators.

End-of-chapter note: `std::bind` always copies its arguments. To bind by reference instead, wrap the argument in `std::ref` (or `std::cref` for const-reference binding).

Chapter 7: The Concurrency API

C++11 incorporates concurrency into both the language (a memory model, `thread_local`, the `std::thread` library, atomics, mutexes, condition variables) and the standard library (`std::future`, `std::shared_future`, `std::async`, `std::promise`, `std::packaged_task`). The items in this chapter favor the higher-level, task-based interfaces over raw threads.

Item 35: Prefer task-based programming to thread-based

Two ways to run a function asynchronously: `std::thread`, which directly creates an OS thread, and `std::async`, which runs the function as a **task** and returns a `std::future` representing its eventual result.

```
std::thread t(doAsyncWork);        // thread-based: low-level
auto fut = std::async(doAsyncWork); // task-based: returns a future
```

Thread-based has no way to retrieve a return value (`std::thread` does not carry one) and propagates exceptions by calling `std::terminate`. Task-based delivers both: `fut.get()` returns the value or rethrows the exception. The task-based interface also relieves you of three responsibilities that `std::thread` forces on you. **Thread exhaustion**: each `std::thread` maps to an OS thread, and creating too many throws `std::system_error`. **Oversubscription**: more software threads ready to run than the hardware can run concurrently causes extra context switching and cache thrashing. **Load balancing**: deciding which work goes on which thread. `std::async` with the default launch policy delegates all of this to the standard library's scheduler. Use `std::thread` directly only when you need the underlying platform API (`pthread_t`, Windows `HANDLE`), are positioned to hand-tune thread usage, or must implement primitives the C++ API does not provide (thread pools, work stealing).

Takeaway: `std::thread` offers no direct way to get return values from asynchronous functions, and unhandled exceptions terminate the program. Thread-based programming requires manual handling of exhaustion, oversubscription, and load balancing. Task-based via `std::async` with the default launch policy handles most of this.

Item 36: Specify `std::launch::async` if asynchronicity is essential

`std::async` takes an optional launch policy from the scoped enum `std::launch`. `launch::async` requires the task to run on a different thread. `launch::deferred` runs the task **synchronously** the first time `get` or `wait` is called on the returned future, and never runs it at all if neither is called. The default policy is `launch::async | launch::deferred`, leaving the choice to the implementation. That flexibility creates three uncertainties: whether the task runs concurrently with the calling thread, whether it runs on the same thread as `get`/`wait` (relevant for `thread_local` access), and whether it runs at all. You also cannot meaningfully time-out a deferred task, since `wait_for`/`wait_until` immediately return `std::future_status::deferred` without doing any waiting; you must test for that explicitly.

```
auto fut = std::async(f);   // default policy

if (fut.wait_for(0s) == std::future_status::deferred) {
  // task is deferred: wait_for/wait_until are useless, must call get()
} else {
  while (fut.wait_for(100ms) != std::future_status::ready) {
    // task is async and not yet done
  }
}
```

The default policy is fine only when concurrency does not matter, `thread_local` identity does not matter, `get`/`wait` is guaranteed to be called (or it is acceptable that the task never runs), and any timeout-based wait loop handles the deferred status. If any of those fails, force concurrent execution.

```
auto fut = std::async(std::launch::async, f);

template<typename F, typename... Ts>
inline auto reallyAsync(F&& f, Ts&&... params) {
  return std::async(std::launch::async,
                    std::forward<F>(f),
                    std::forward<Ts>(params)...);
}
```

Takeaway: The default launch policy permits both asynchronous and synchronous execution. This flexibility creates uncertainty around `thread_local` access, implies the task may never run, and complicates timeout-based wait loops. Specify `std::launch::async` when asynchronous execution is essential.

Item 37: Make `std::thread`s unjoinable on all paths

A `std::thread` is **joinable** if it corresponds to an underlying thread of execution that is or could be running (including blocked or waiting threads). It is **unjoinable** if it was default-constructed, moved-from, already joined, or already detached. Destroying a **joinable** `std::thread` calls `std::terminate`. The standard chose this over the two natural alternatives because both were worse. **Implicit join** on destruction would silently wait for the thread to finish at the closing brace, producing hard-to-diagnose stalls. **Implicit detach** would sever the connection but let the thread continue, leaving it running over destroyed stack frames, the textbook recipe for undefined behavior. Termination is loud, so the standard picked loud. The consequence is that any `std::thread` member or local must be made unjoinable on every exit path, normal or exceptional. Wrap it in an RAII type that joins (or detaches, if that is correct for the use case) in its destructor.

```
class ThreadRAII {
public:
  enum class DtorAction { join, detach };
  ThreadRAII(std::thread&& t, DtorAction a) : action(a), t(std::move(t)) {}
  ~ThreadRAII() {
    if (t.joinable()) {
      if (action == DtorAction::join) t.join();
```

```
      else                                t.detach();
    }
  }
  ThreadRAII(ThreadRAII&&)            = default;
  ThreadRAII& operator=(ThreadRAII&&) = default;
  std::thread& get() { return t; }
private:
  DtorAction action;
  std::thread t;          // declared LAST: data members
used by the
                          // destructor must be alive
when it runs
};
```

The `std::thread` member is declared last on purpose: members are destroyed in reverse declaration order, and a destructor running on a joinable thread member must finish before any member it depends on is gone.

Takeaway: Make `std::thread`s unjoinable on all paths. Join-on-destruction can produce hard-to-debug stalls; detach-on-destruction can produce undefined behavior. Declare `std::thread` data members last so they outlive the members their destructor logic might touch.

Item 38: Be aware of varying thread handle destructor behavior

A `std::future` is a handle to the result of an asynchronous operation. The result has to live somewhere both producer and consumer can see, and that location, the **shared state**, is typically heap-allocated. Most future destructors are unremarkable: they destroy the future's members and decrement the shared state's reference count, no blocking. The exception: the destructor of the **last** future referring to a shared state that was created by `std::async` for a `launch::async` task **blocks until the task completes**. It is effectively an implicit join. For deferred tasks where this future is the last reference, the task simply never runs.

```
{
  auto fut = std::async(std::launch::async,
longRunning);
  // fut goes out of scope here, dtor blocks until
longRunning finishes
}
```

Three conditions must all hold for the blocking behavior: the shared state arose from `std::async` (not `std::packaged_task` or `std::promise`), the launch policy was `launch::async`, and this is the last future referring to the state. The API gives no programmatic way to ask whether a future is in that state, which makes the implicit-join behavior easy to miss. `std::packaged_task` yields a future whose destructor is always the normal non-blocking kind.

Takeaway: Future destructors normally just destroy the future's members. The final future referring to a shared state for a non-deferred task launched via `std::async` blocks until the task completes.

Item 39: Consider `void` futures for one-shot event communication

For "task A signals task B that something happened," the obvious tool is `std::condition_variable`. The standard idiom requires a mutex (even when no data is being protected) and a predicate to handle spurious wakeups; it also fails outright if the reactor has not yet entered the wait when the detector calls `notify_one`.

```
std::condition_variable cv;
std::mutex m;
bool flag = false;

// detector:
{ std::lock_guard<std::mutex> g(m); flag = true; }
cv.notify_one();

// reactor:
{ std::unique_lock<std::mutex> lk(m);
  cv.wait(lk, []{ return flag; }); }  // lambda handles
spurious wakeups
```

A pure atomic flag with polling avoids the mutex but burns CPU and is not truly blocked. A cleaner one-shot is a `std::promise<void>` paired with the `std::future<void>` it produces. The reactor waits on the future; the detector fulfills the promise to release the wait. No mutex, no spurious-wakeup logic, no ordering constraint between the two tasks, and the reactor truly blocks.

```
std::promise<void> p;

// reactor (waits):
auto fut = p.get_future();
// ...
fut.wait();        // truly blocks until the promise is
set
react();

// detector (signals):
p.set_value();     // releases the wait
```

Costs: the shared state is heap-allocated, and a promise can only be set once, so this is a one-shot channel, not a reusable signal. A useful application is suspending a `std::thread` immediately after construction so the caller can configure it before it runs.

Takeaway: Condvar-based event signaling requires a superfluous mutex, imposes ordering constraints between detector and reactor, and forces the reactor to verify the event. Flag-based designs avoid those problems but poll instead of block. `std::promise<void>` plus `std::future<void>` dodges both, at the cost of heap-allocated shared state and a one-shot lifetime.

Item 40: Use `std::atomic` for concurrency, `volatile` for special memory

The two are unrelated despite both being called "volatile-ish" by people who do not use them. `std::atomic` makes inter-thread access well-defined: reads and writes are indivisible, and the compiler is forbidden from reordering ordinary memory accesses across them in ways that would be visible to other threads. `volatile` tells the compiler "do not optimize accesses to this memory" because the memory does not behave normally (memory-mapped I/O, signal handlers, hardware registers): no dead-store elimination, no read coalescing, no caching the value in a register. `volatile` provides **no** atomicity and **no** inter-thread ordering guarantees.

```
std::atomic<int> ai{0};
ai = 10;           // atomic store
std::cout << ai;   // atomic load (the read is atomic;
```

```
                    // the whole expression is not one
critical section)
int x = ++ai;       // atomic read-modify-write

volatile int vi = 0;
++vi;               // NOT atomic; just "do the read and
the write,
                    // do not optimize them away"
```

Compound operations like `++ai` are atomic on `std::atomic`; on `volatile` they decompose into a separate read and write that another thread can race against. `std::atomic`'s copy operations are deleted (a single atomic read-and-write of two separate objects is not implementable in general hardware); use `load` and `store` member functions if you want the atomic access spelled out explicitly, which also flags the access for human readers as a synchronization point. The two combine sensibly when both properties are needed: a `volatile std::atomic<int>` is appropriate for a memory-mapped register accessed concurrently by threads, where you need the compiler to leave the accesses alone **and** multi-threaded atomicity.

Takeaway: `std::atomic` is for data accessed from multiple threads without a mutex; it is a tool for concurrent code. `volatile` is for memory whose reads and writes must not be optimized away; it is a tool for special memory. Neither substitutes for the other.

Chapter 8: Tweaks

Item 41: Consider pass-by-value for copyable parameters that are cheap to move and always copied

The canonical setter takes its argument by `const T&` and copies; an optimized modern version takes it by universal reference and forwards; the pass-by-value form is the middle ground.

```
class Widget {
public:
  void addName(std::string newName) {
    names.push_back(std::move(newName));   // newName is
local, move is safe
  }
private:
  std::vector<std::string> names;
};
```

For an lvalue argument the parameter is **copy-constructed** and then **moved** into the vector: one copy plus one move. For an rvalue argument the parameter is **move-constructed** and then **moved** into the vector: two moves. Compared with the two overloaded forms (`const T&` doing a copy, `T&&` doing a move), this costs one extra move per call, which is acceptable when moves are cheap. The upside is one function instead of two, and no template machinery. Three preconditions for this to be a good idea: the parameter is **copyable** (so the by-value form is well-formed even for lvalue arguments), moves on it are **cheap**, and the function **always copies** the argument (otherwise the extra move is pure waste). Two cautions. First, copying via construction is often more expensive than copying via assignment, because assignment can reuse the destination's existing allocation (a `std::string`'s buffer) while construction must allocate fresh. If `names.push_back` would have triggered an assignment, the pass-by-value version forces a construction and you lose. Second, pass-by-value **slices**: a derived-class argument passed by value to a function taking the base by value is sliced down to the base, so do not use this pattern when the parameter type is a base class likely to be called with derived arguments.

> **Takeaway:** For copyable, cheap-to-move parameters that are always copied, pass-by-value can approach the efficiency of by-reference, is easier to implement, and can produce less object code. Copying via construction can be significantly more expensive than copying via assignment. Pass-by-value is subject to slicing, so avoid it for base class parameter types.

Item 42: Consider emplacement instead of insertion

`push_back` (and `insert`, `push_front`) takes an **object** of the container's element type. `emplace_back` (and `emplace`, `emplace_front`) takes **constructor arguments** and constructs the element in place inside the container, by perfect-forwarding the arguments to the element's constructor. The win is avoiding a temporary.

```
std::vector<std::string> vs;
vs.push_back("xyzzy");      // 1. ctor a temp
std::string from the literal
                            // 2. move-construct into
the vector
                            // 3. destroy the temp
vs.emplace_back("xyzzy");   // construct the std::string
directly in place
vs.emplace_back(50, 'x');   // even cleaner: builds
string of 50 'x's in place
```

Emplacement is available on every standard container that supports the matching insert family: `emplace_back` where `push_back` exists, `emplace_front` where `push_front` exists, `emplace` where `insert` exists (every container except `forward_list` and `array`); associative containers also offer `emplace_hint`, and `forward_list` has `emplace_after`. Three conditions make emplacement faster than insertion in practice: the value is **constructed** into the container rather than assigned (i.e. the slot does not already hold an object), the argument type differs from the container's element type (so insertion would build a temporary that emplacement skips), and the container is unlikely to reject the value as a duplicate (since duplicate detection runs after the in-place construction, which has already done work). **Exception-safety trap with resource-owning elements** The advantage flips on its head when the element is a resource manager built from a raw pointer. With `push_back`, the `shared_ptr` is constructed **before** the call, taking ownership of the raw pointer; if `push_back` then fails (OOM on a reallocation), the `shared_ptr`'s destructor frees the resource.

```
std::list<std::shared_ptr<Widget>> ptrs;

ptrs.push_back(std::shared_ptr<Widget>(new Widget,
customDel));  // safe
// raw new Widget is wrapped immediately; if push_back
throws,
// the shared_ptr's dtor cleans up.

ptrs.emplace_back(new Widget,
customDel);                          // leaks on OOM
// emplace_back forwards the raw pointer + deleter into
the
// shared_ptr ctor *inside* the container. If list
allocation
// fails between receiving the raw pointer and that
ctor, the
// raw new Widget leaks.
```

The point generalizes: emplacement defers the construction of the resource-managing object to the last moment, opening a window between "resource acquired" and "resource owned" where an exception leaks. Prefer insertion (or construct the resource manager into a named local first, then emplace by `std::move`) for `unique_ptr` / `shared_ptr` elements. A side effect of the by-construction-arguments interface: emplacement can perform conversions that insertion would reject, including some `explicit` conversions. Useful when you want the conversion; a footgun when you do not.

> **Takeaway:** Emplacement should sometimes outperform insertion and should never underperform it. Practical wins come when the value is constructed (not assigned) into the container, the argument types differ from the element type, and duplicates are unlikely. Emplacement performs type conversions insertion would reject,

> including the exception-unsafe case of forwarding raw pointers into `shared_ptr` / `unique_ptr` elements.